

AhamX Bodhi

Where I Becomes Infinite

Encoder-Decoder Architecture

Module 3.2: Transformer Architecture Deep-Dive
Prof. Sashikumaar Ganesan | IISc Bengaluru



Session Overview

What You Will Learn

- The original encoder-decoder design from 2017
- What the encoder and decoder each do
- The role of cross-attention in connecting them
- When to use encoder-decoder vs. decoder-only
- Production models: T5, BART, mT5, Whisper

Concept at a Glance

- **Duration:** 9 minutes
- **Bloom's Level:** B2 – Understand
- **Difficulty:** L2 – Intermediate
- **Module:** 3.2 Transformer Deep-Dive
- **Prerequisite:** Scaled Dot-Product Attention



Learning Objectives

By the End of This Session You Will Be Able To

1. **Describe** the encoder stack and decoder stack and the role of each
2. **Explain** how cross-attention connects encoder outputs to the decoder
3. **Distinguish** encoder-decoder from encoder-only and decoder-only models
4. **Identify** three production model families that use encoder-decoder design
5. **Select** the appropriate architecture family for a given NLP task



Prerequisites and Connections

You Should Already Know

- Self-attention and the QKV abstraction
- Scaled dot-product attention formula
- Multi-head attention concept
- The three attention variants from the original paper

This Concept Unlocks

- Decoder-Only Architecture (comparison)
- Seq2Seq task implementation
- Whisper: speech-to-text architecture
- Cross-attention in multimodal models





The Encoder Stack

What the Encoder Does

- Receives the full source sequence (e.g. source language sentence)
- Each encoder layer contains:
 - **Multi-Head Self-Attention:** Every source token attends to every other source token — full bidirectional context
 - **Feed-Forward Network:** Applied independently to each token position
 - **Residual connections and Layer Norm** wrap each sub-layer
- Output: A rich contextual representation for **every source token**, shape $(n_{\text{src}} \times d_{\text{model}})$
- The **entire source sequence is processed in parallel** — no sequential steps

The encoder transforms raw token embeddings into semantically rich representations that the decoder will query during generation.



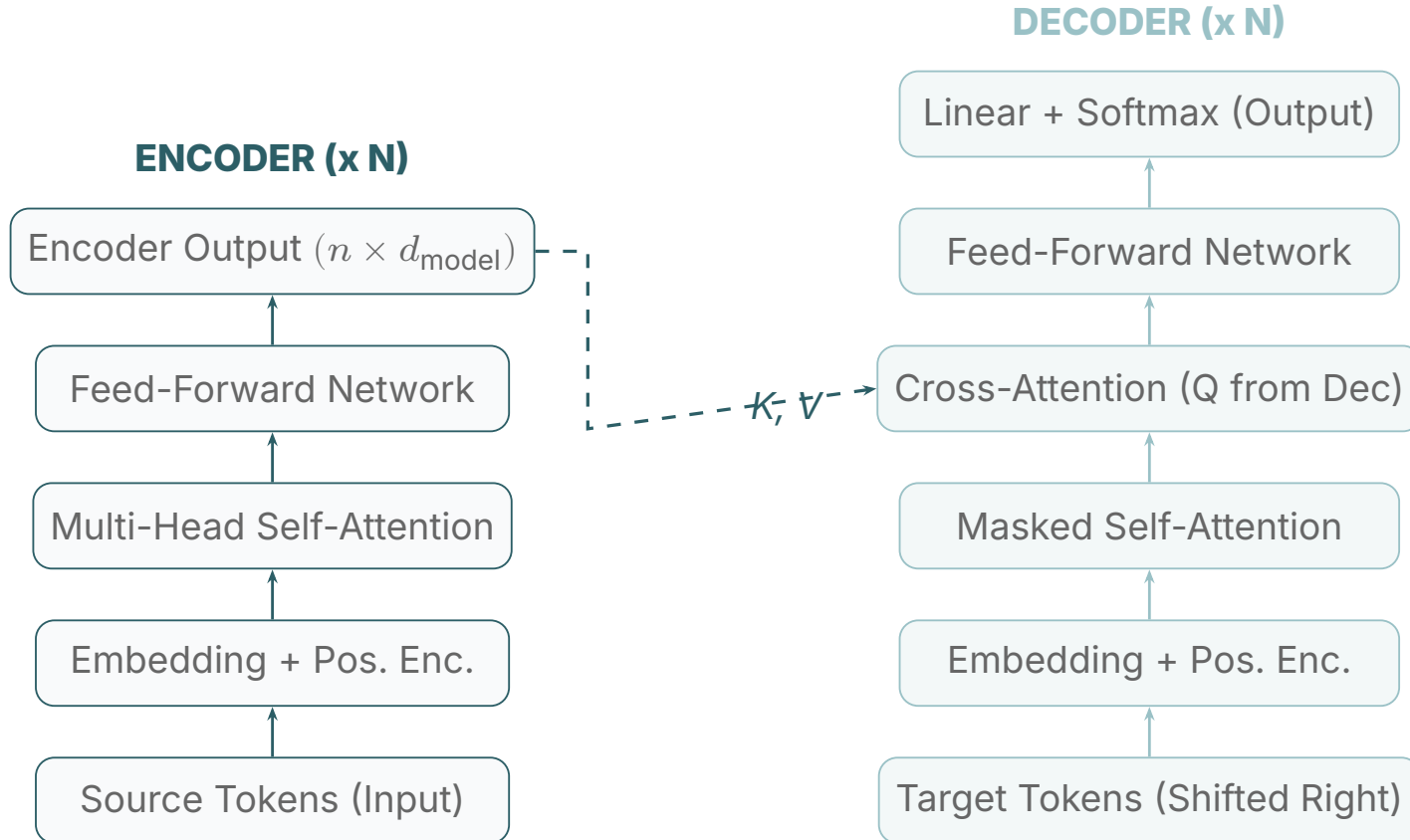
The Decoder Stack

What the Decoder Does

- Generates the target sequence token by token (autoregressive)
- Each decoder layer contains **three** sub-layers:
 - **Masked Multi-Head Self-Attention:** Decoder tokens attend only to prior decoder positions (causal masking enforced)
 - **Cross-Attention:** Decoder queries (Q) attend to encoder keys and values (K, V) — this is how source context is used
 - **Feed-Forward Network:** Per-position transformation
- Every sub-layer wrapped with residual connection and Layer Norm
- At training: all target tokens processed in parallel (teacher forcing)
- At inference: tokens generated one at a time, feeding back as input



Encoder-Decoder Architecture Flow





Cross-Attention: The Bridge Between Encoder and Decoder

How the Decoder Reads the Source

- In cross-attention: **Q comes from the decoder** (what the decoder wants to know), **K and V come from the encoder** (what the source provides)
- For each decoder position, cross-attention computes which source tokens are most relevant and blends their value vectors accordingly
- This is mathematically identical to scaled dot-product attention — only the *source* of Q vs. K/V differs
- The encoder is run **once**; its K and V outputs are cached and reused at every decoder step during inference

Cross-attention is the alignment mechanism: it learned to align “bank” in French to “bank” in English without explicit supervision.



Architecture Family Comparison

Feature	Encoder-Only	Encoder-Decoder	Decoder-Only
Attention	Bidirectional	Bi-enc + Causal-dec	Causal
Primary task	Classification, NER	Translation, Summ.	Text generation
Key models	BERT, RoBERTa	T5, BART, Whisper	GPT-4, Claude, Llama
Context	Full source	Source + target	Prior tokens only
Parallelism	Full	Partial (dec is auto.)	Partial (dec is auto.)
Parameter ratio	1x	~2x	1x



T5: Encoder-Decoder for Unified NLP

Text-to-Text Transfer Transformer (Raffel et al., 2020)

- **Design:** Every NLP task reformulated as text → text: "translate English to German: ...", "summarise: ...", "answer: ..."
- **Architecture:** 12-layer encoder, 12-layer decoder; pre-trained on C4 (750GB cleaned Common Crawl)
- **Scale:** T5-Small (60M) to T5-XXL (11B); all share the same architecture
- **Variants:** mT5 (multilingual), FLAN-T5 (instruction-tuned), UL2 (mixture-of-denoisers) — all use encoder-decoder
- **In production:** Google uses T5-family models for search query understanding, featured snippets, and Gmail Smart Reply

T5 proved that encoder-decoder with a unified interface can match task-specific models.





GenAI Application: Neural Machine Translation

The Task That Defined the Architecture

- **Google Translate:** Encoder reads source sentence in any of 100+ languages; decoder generates the translation token by token
- **DeepL:** Uses custom encoder-decoder models trained on high-quality curated parallel corpora; outperforms general-purpose models on document translation
- **Meta NLLB-200:** No Language Left Behind — encoder-decoder covering 200 languages including low-resource languages with no prior model coverage
- **Real-time subtitling:** YouTube auto-caption pipeline: Whisper (encoder-decoder ASR) → NLLB-200 (encoder-decoder MT) in sequence



GenAI Application: Summarisation and Document Processing

Encoder-Decoder for Conditional Text Generation

- **BART (Meta):** Encoder-decoder pre-trained with denoising objectives; state-of-the-art for news summarisation (CNN/DailyMail) and paraphrase generation
- **Pegasus (Google):** Encoder-decoder specifically pre-trained for summarisation with gap-sentence generation; used in Google Docs “summarise” feature
- **Legal and medical summarisation:** Encoder reads 50-page contracts or clinical notes; decoder generates structured summaries with key clauses identified
- **Multi-document summarisation:** Encoder processes multiple retrieved documents; decoder synthesises a coherent summary — core of many RAG pipelines



GenAI Application: Speech-to-Text with Whisper

OpenAI Whisper — Encoder-Decoder for Audio

- **Architecture:** Audio spectrogram → Convolutional front-end → Transformer encoder → Transformer decoder → transcribed text
- **Cross-attention role:** Decoder attends to encoded audio features at each generation step — the same mechanism as in NMT
- **Scale:** Trained on 680K hours of multilingual audio; 99 languages supported; near-human word error rates for English
- **In production:** Otter.ai, Zoom, Google Meet, Apple Siri, and hundreds of transcription services use Whisper or fine-tuned variants

Whisper demonstrates that the encoder-decoder architecture generalises beyond text-to-text.



GenAI Application: Structured Output Generation

Encoder-Decoder for Constrained Generation Tasks

- **SQL generation from NL:** Encoder reads the schema and natural language query; decoder generates syntactically valid SQL — used in Databricks, Snowflake
- **Code generation from specifications:** Encoder reads a function specification; decoder generates the implementation — used in AlphaCode for competitive programming
- **Question answering:** Encoder reads context + question; decoder generates an abstractive answer (not just extraction) — powers Google's featured snippets
- **Data-to-text generation:** Encoder processes structured tables (weather, sports); decoder generates fluent natural language descriptions for news automation



When to Choose Encoder-Decoder

Task Characteristics That Favour This Architecture

- Source and target are in **different modalities or languages** (speech → text, French → English)
- The output is significantly **shorter or restructured** relative to the input (summarisation, abstractive QA)
- **Precise cross-referencing** between source and target is required (the decoder needs to “look up” specific parts of the source)
- The task requires **full source context** before any target token is generated

Use Decoder-Only When

The task is open-ended generation, chat, or reasoning where no explicit source document needs to be aligned with the output.



Common Misconceptions

Clearing Up Confusion About Encoder-Decoder Models

- **"Encoder-decoder is the standard for modern LLMs"**: Most production LLMs (GPT-4, Claude, Llama) are decoder-only. Encoder-decoder is preferred for seq2seq tasks with clear source/target distinction.
- **"The encoder produces a single context vector"**: In Transformer encoder-decoder, the encoder produces one contextual vector *per source token* — not a single compressed vector (that was the old RNN approach).
- **"Cross-attention is the same as self-attention"**: They use the same formula, but cross-attention draws Q and K/V from *different* sequences — a crucial architectural distinction.



Encoder-Decoder Architecture — Key Takeaways

1. The encoder processes the full source sequence bidirectionally, producing contextual representations for every source token
2. The decoder generates the target sequence autoregressively using masked self-attention and cross-attention to the encoder outputs
3. Cross-attention is the bridge: Q from decoder, K/V from encoder — enabling the decoder to “align” with relevant source positions
4. Encoder-decoder excels at seq2seq tasks: translation, summarisation, speech recognition, structured generation
5. T5, BART, and Whisper are the dominant production encoder-decoder models; most chat LLMs use decoder-only for simplicity at scale



What This Concept Unlocks

Next in Module 3.2

- Decoder-Only Architecture
- Layer Normalisation
- Feed-Forward Networks in Transformers
- Full Transformer Implementation (coding)

Later in the Course

- Cross-Attention in Diffusion Models
- Seq2Seq Fine-tuning Strategies
- Whisper Fine-tuning for Custom Domains
- Multimodal Architectures (vision + language)

Thank You

Encoder-Decoder Architecture | Module 3.2 | AhamX Bodhi